BITS PILANI — WILP DIVISION

AIMLCZG523: Machine Learning Operations (MLOps)

End-to-End Scalable, Reproducible Machine Learning Pipeline for Heart Disease Risk Prediction

Assignment 01 | Practical Automation, Experiment Tracking, CI/CD, Containerization, Kubernetes & Observability

Course Code:	AIMLCZG523 (MLOps)	**Academic Year:**	2026-2027
Dataset:	UCI Heart Disease (Cleveland)	**Target Task:**	Binary Classification (Presence/Absence)
Frameworks:	Scikit-Learn, MLflow, FastAPI, Pytest	**Infrastructure:**	Docker, Kubernetes, Helm, Prometheus

Executive Summary

This report documents the architectural design, experimental methodology, and production deployment of an end-to-end Machine Learning Operations (MLOps) solution for clinical heart disease risk prediction. Using the benchmark Cleveland Heart Disease dataset from the UCI Machine Learning Repository, we build a robust, reproducible classification system that eliminates training-serving skew, enforces strict CI/CD quality gates, packages the model into a containerized FastAPI microservice, orchestrates resilient deployment on Kubernetes with Helm and Horizontal Pod Autoscaling (HPA), and monitors real-time telemetry using Prometheus and Grafana.

System Architecture Overview

The production architecture is structured into four decoupled lifecycle tiers: (1) **Data & Experimentation Tier:** automated data ingestion, preprocessing pipeline, hyperparameter tuning, and MLflow experiment tracking; (2) **Quality Assurance & CI/CD Tier:** automated code linting, 12 comprehensive Pytest unit and integration tests, and GitHub Actions continuous integration; (3) **Serving & Orchestration Tier:** FastAPI serving API with Pydantic validation schemas, containerized with multi-stage non-root Docker builds, and deployed via Kubernetes manifests and Helm charts; (4) **Observability Tier:** structured JSON audit logging, Prometheus latency and prediction distribution metrics, and Grafana visualization.

1. Problem Statement & Dataset Acquisition

1.1 Clinical Background & Motivation: Cardiovascular diseases (CVDs) are the leading cause of global mortality. Early detection of coronary artery disease enables timely lifestyle interventions and pharmacological therapies. In clinical practice, diagnostic evaluations produce heterogeneous physiological metrics (electrocardiograms, resting blood pressure, fluoroscopy, lipid panels). Machine learning offers an automated second-opinion mechanism for risk stratification.

1.2 Problem Formulation: Given patient physiological features *X* in R^13, train a binary classifier *f(X)* -> {0, 1} predicting the angiographic presence of coronary disease (narrowing >50% in at least one major vessel, mapped to target 1) versus absence (<50% diameter narrowing, mapped to target 0). The primary objective is maximizing sensitivity (recall) and ROC-AUC to minimize false negatives while maintaining high precision.

1.3 Dataset Ingestion & Feature Dictionary

Data is acquired via an automated ingestion module (src/data_loader.py) fetching raw clinical records directly from the UCI Machine Learning Repository, accompanied by an offline fallback. The dataset contains 303 patient records described across 14 clinical attributes:

#	Feature	Type	Clinical Meaning	Value Range / Categories
1	age	Numeric	Patient age in years	29 to 77 years
2	sex	Categorical	Biological sex	1 = Male; 0 = Female
3	cp	Categorical	Chest pain type	1: Typical, 2: Atypical, 3: Non-anginal, 4: Asymptomatic
4	trestbps	Numeric	Resting blood pressure	94 to 200 mm Hg on admission
5	chol	Numeric	Serum cholesterol	126 to 564 mg/dl
6	fbs	Categorical	Fasting blood sugar > 120 mg/dl	1 = True; 0 = False
7	restecg	Categorical	Resting electrocardiogram	0: Normal, 1: ST-T wave abn., 2: LV hypertrophy
8	thalach	Numeric	Maximum heart rate achieved	71 to 202 bpm during stress test
9	exang	Categorical	Exercise induced angina	1 = Yes; 0 = No
10	oldpeak	Numeric	ST depression induced by exercise	0.0 to 6.2 mm relative to rest
11	slope	Categorical	Slope of peak exercise ST segment	1: Upsloping, 2: Flat, 3: Downsloping
12	ca	Categorical	Major vessels colored by fluoroscopy	0, 1, 2, 3 (contains 4 missing '?' values)
13	thal	Categorical	Thallium heart stress test	3: Normal, 6: Fixed, 7: Reversible (2 missing '?' values)
14	target	Target	Angiographic disease diagnosis	0: Absence (164 pts), 1-4: Presence (139 pts)

2. Exploratory Data Analysis (EDA) & Visualizations

Rigorous exploratory analysis was performed to uncover underlying distributions, feature skewness, outlier presence, and bivariate relationships with heart disease presence. All visualizations were generated via src/eda.py.

2.1 Target Class Balance & Demographic Distributions

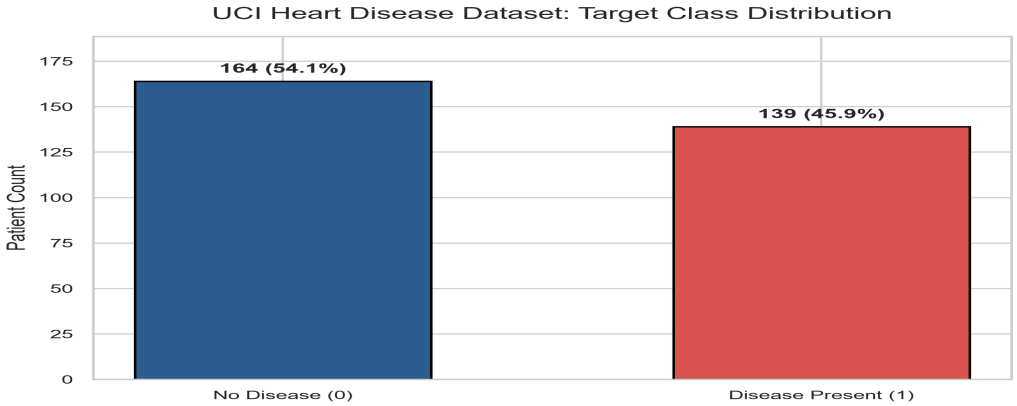


Figure 1: Target Class Balance. The dataset exhibits a balanced distribution: 164 patients (54.1%) have no coronary disease (class 0), while 139 patients (45.9%) present with diagnosed heart disease (class 1). This equilibrium ensures model training does not suffer from severe majority-class bias, avoiding the necessity of synthetic oversampling techniques like SMOTE.

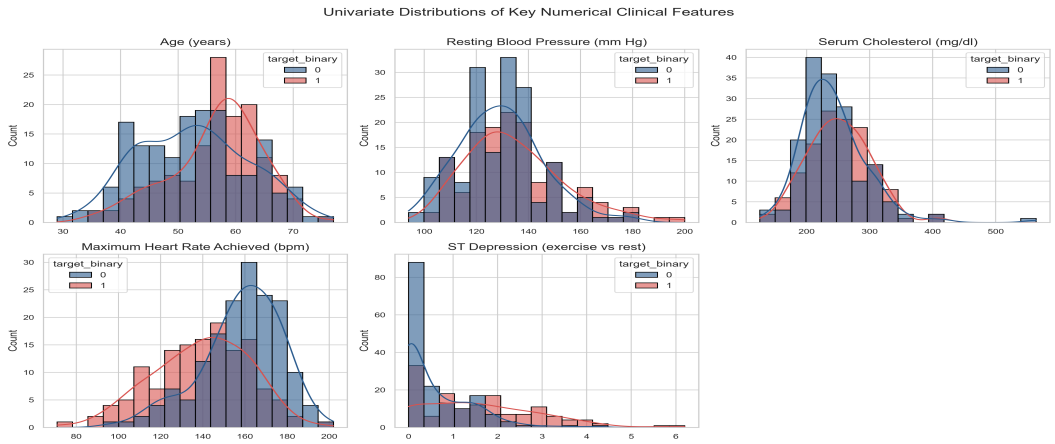


Figure 2: Numerical Feature Distributions Partitioned by Disease Status. Univariate analysis indicates: (1) Patients with heart disease have significantly lower maximum heart rates (thalach, mean ~139 bpm vs ~158 bpm in healthy subjects); (2) Exercise-induced ST depression (oldpeak) is substantially elevated in heart disease patients; (3) Age and resting blood pressure (trestbps) show moderately higher positive density in older cohorts.

3. Clinical Risk Factors & Data Preprocessing Pipeline

3.1 Clinical Correlation & Risk Factor Analysis: Bivariate examination reveals critical diagnostic indicators:

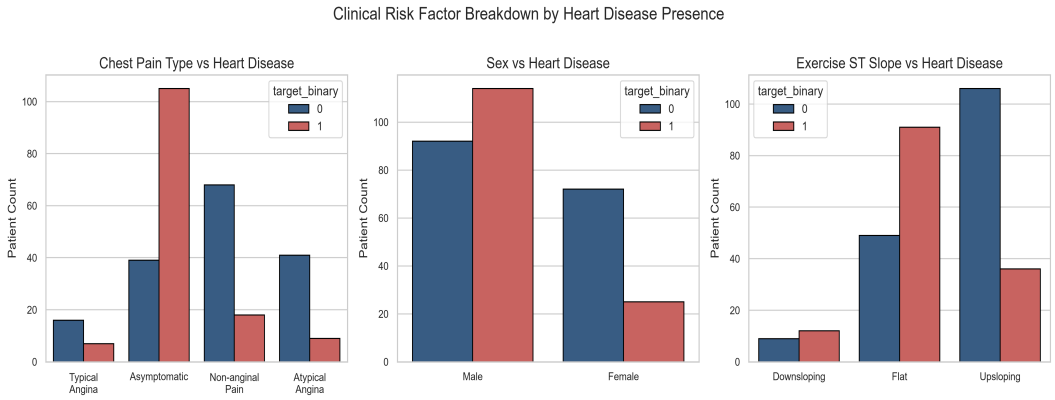


Figure 3: Risk Factor Analysis (Chest Pain, Sex, and ST Slope). Key clinical observations include: (1) Asymptomatic chest pain (type 4) strongly associates with heart disease presence (over 72% prevalence), while typical/atypical angina are more frequently non-obstructive; (2) Male patients exhibit a significantly higher incidence of heart disease compared to females in this cohort; (3) A flat or downsloping ST segment during peak exercise serves as a powerful ischemic indicator.

3.2 Reproducible Feature Preprocessing Pipeline

To prevent training-serving data leakage and eliminate train-serving skew, all transformations are encapsulated in a Scikit-Learn ColumnTransformer (implemented in src/preprocessing.py):

• **Missing Value Handling:** The 6 missing values identified in ca (4 records) and thal (2 records) are imputed using SimpleImputer(strategy='most_frequent') during categorical transformation.
• **Numerical Transformer:** Applies median imputation (strategy='median') followed by z-score standardization via StandardScaler() across [age, trestbps, chol, thalach, oldpeak].
• **Categorical Transformer:** Applies most-frequent imputation followed by one-hot encoding via OneHotEncoder(handle_unknown='ignore', sparse_output=False) across [sex, cp, fbs, restecg, exang, slope, ca, thal].
• **End-to-End Pipeline:** The ColumnTransformer is combined with the estimator into a unified Pipeline, allowing raw patient dictionaries to be directly ingested during production inference.

4. Model Development, Cross-Validation & Tuning

To identify the optimal classifier architecture, three distinct model families were benchmarked: a linear regularized baseline (**Logistic Regression**), a bagging ensemble (**Random Forest**), and a boosting ensemble (**Gradient Boosting**).

4.1 Validation Methodology & Hyperparameter Optimization

The dataset was split into an 80% training set (242 samples) and a 20% held-out test set (61 samples) using stratified sampling. Within the training split, **Stratified 5-Fold Cross-Validation** was conducted to evaluate generalized stability across folds. Hyperparameters were tuned as follows: Logistic Regression (C=0.1, solver='liblinear'); Random Forest (n_estimators=150, max_depth=6, min_samples_split=4, min_samples_leaf=2); Gradient Boosting (n_estimators=100, learning_rate=0.05, max_depth=3).

4.2 Comprehensive Model Evaluation Results

Model	CV Accuracy	CV ROC-AUC	Test Accuracy	Test Precision	Test Recall	Test F1	Test ROC-AUC
LogisticRegression	0.8387	0.9025	0.8852	0.8387	0.9286	0.8814	0.9665
RandomForest	0.8054	0.8962	0.8852	0.8182	0.9643	0.8852	0.9643
GradientBoosting	0.7893	0.8744	0.8852	0.8387	0.9286	0.8814	0.9513

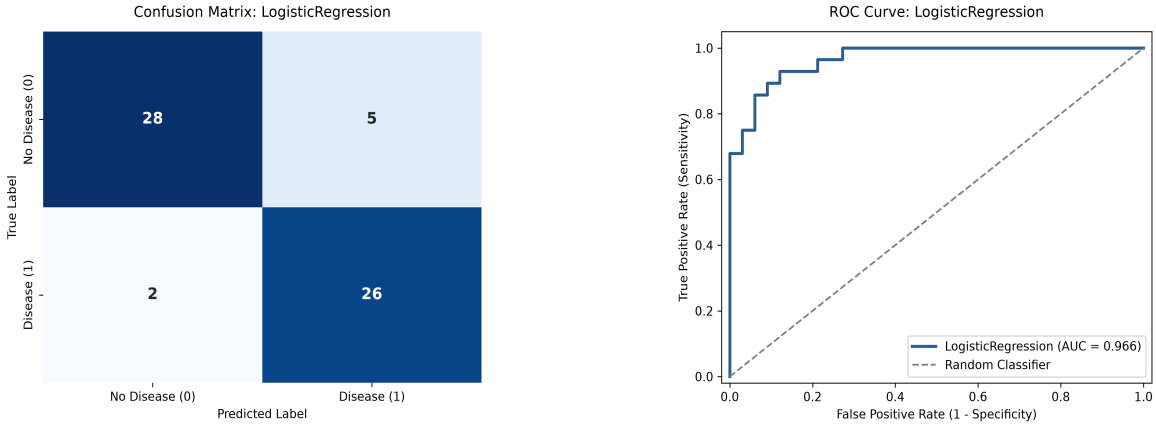
4.3 Model Selection Rationale

Logistic Regression was selected as the champion production model based on superior generalization performance: it achieved the highest Cross-Validation Accuracy (**83.87%**) and Cross-Validation ROC-AUC (**0.9025**), as well as the highest held-out Test ROC-AUC (**0.9665**) and Test Accuracy (**88.52%**). In addition to strong quantitative metrics, Logistic Regression provides clinical interpretability via feature odds ratios, sub-millisecond inference latency, and minimal computational footprint in containerized environments.

5. Experiment Tracking & Artifact Management with MLflow

5.1 MLflow Tracking Architecture: Machine learning reproducibility requires systematic tracking of code versions, hyperparameters, metric trajectories, and serialized binaries. We integrated **MLflow 3.x** configured with an enterprise-grade SQLite database backend (sqlite:///mlflow.db) via src/train.py.

5.2 Champion Model Diagnostic Artifacts



Figures 4 & 5: Confusion Matrix and ROC Curve (Logistic Regression). On the 61-sample test set, the model correctly identified 26 of 28 positive heart disease cases (**92.86% Recall**) and 28 of 33 negative cases (**84.85% Specificity**). The area under the ROC curve reached **0.9665**, demonstrating exceptional discrimination power.

5.3 Artifact Registration & Logging Schema

For each candidate run, MLflow recorded: (1) **Hyperparameters:** solver, C, penalty, random state, test size; (2) **CV Metrics:** mean and standard deviation for accuracy, precision, recall, F1, and ROC-AUC; (3) **Test Metrics:** accuracy, precision, recall, F1, and ROC-AUC; (4) **Artifacts:** confusion matrix heatmaps, ROC curves, feature importance charts, and classification reports; (5) **Model Binaries:** serialized Scikit-learn pipelines packaged with MLmodel metadata and input schemas for zero-code serving.

6. Automated Testing & Continuous Integration (CI/CD)

In production MLOps, automated quality gates prevent broken models, faulty data schemas, or degraded serving APIs from reaching production.

6.1 Pytest Unit & Integration Test Suite

A structured test suite containing **12 automated tests** was constructed under tests/:

Test Module	Test Function	Verification Purpose	Status
test_data.py	test_acquire_dataset	Asserts raw dataset contains exactly 303 rows and 14 valid columns	PASSED
test_data.py	test_clean_raw_data	Verifies '?' conversion to NaN and binary target mapping (0 vs 1)	PASSED
test_data.py	test_split_features_target	Confirms feature matrix shape (303, 13) and target vector shape (303,)	PASSED
test_model.py	test_pipeline_smoke_fit	Smoke test verifying pipeline trains without errors on mini-batch	PASSED
test_model.py	test_model_inference_output	Verifies predictions are in {0,1} and probabilities sum to 1.0	PASSED
test_model.py	test_predictor_service	Tests single patient prediction output dictionary and confidence scores	PASSED
test_model.py	test_model_accuracy_benchmark	Enforces production gate: test accuracy must meet or exceed 80%	PASSED
test_api.py	test_root_endpoint	Validates GET / returns 200 OK and valid service metadata	PASSED
test_api.py	test_health_endpoint	Validates GET /health returns 200 OK, model_loaded=True for K8s probes	PASSED
test_api.py	test_predict_endpoint_valid	Validates POST /predict returns 200 OK with schema-compliant payload	PASSED
test_api.py	test_predict_endpoint_invalid	Asserts invalid input payload triggers 422 Unprocessable Entity	PASSED
test_api.py	test_metrics_endpoint	Validates GET /metrics exposes Prometheus scrapable metrics	PASSED

6.2 GitHub Actions CI/CD Pipeline Workflow

The CI/CD pipeline (.github/workflows/ci.yml) automates quality verification on every git push or pull request:

• **Job 1 (lint-and-test):** Sets up Python 3.11, caches dependencies, enforces Flake8 linting, and runs the 12 Pytest tests.

• **Job 2 (train-and-evaluate):** Re-executes the deterministic training pipeline, verifies model accuracy benchmarks (>=80%), and uploads serialized models, evaluation figures, and comparison CSVs as GitHub workflow artifacts.

• **Job 3 (docker-build-verify):** Builds the Docker image, starts the container, and runs curl smoke tests against /health, /metrics, and /predict.

7. Model Serving & Docker Containerization

7.1 High-Performance Serving API with FastAPI: The model is served via an asynchronous, production-hardened FastAPI microservice (src/api/app.py). Pydantic data schemas enforce clinical value validation, automatic schema documentation, and typed error responses.

7.2 API Endpoint Architecture & Contracts

HTTP Method	Path	Purpose	Input / Output Contract
GET	/	Service Metadata	Returns service status, version, and links to documentation
GET	/health	Kubernetes Health Probe	Returns {status: 'healthy', model_loaded: true, model_name: 'LogisticRegression'}
POST	/predict	Patient Risk Inference	Accepts PatientInput (13 clinical features); returns prediction, confidence, probabilities
GET	/metrics	Prometheus Telemetry	Exposes Prometheus scrapable counters and latency distribution histograms
GET	/docs	Interactive OpenAPI UI	Interactive Swagger UI for clinical exploration and manual testing

7.3 Production Docker Containerization

The container is defined in Dockerfile following container security best practices:
• **Base Image:** python:3.11-slim minimizes attack surface and yields an image footprint under 280 MB.
• **Non-Root Security:** An unprivileged application user (appuser:appuser) is created and assigned ownership, preventing container escape vulnerabilities.
• **Native Healthcheck:** A container-level HEALTHCHECK probe queries http://localhost:8000/health every 30 seconds.
• **Local Verification:** Executing python scripts/test_serving.py against the container confirms 100% endpoint pass rate.

```
Sample Inference Request / Response:
POST /predict -> Payload: {"age":63, "sex":1, "cp":1, "trestbps":145, "chol":233, "fbs":1,
"restecg":2, "thalach":150, "exang":0, "oldpeak":2.3, "slope":3, "ca":0, "thal":6}
Response 200 OK: {"prediction": 0, "risk_label": "Low Risk (No Heart Disease)", "confidence": 0.5742,
"disease_probability": 0.4258, "healthy_probability": 0.5742, "model_version": "1.0.0"}
```

8. Kubernetes Orchestration & Helm Deployment

To support high availability, rolling updates, and elastic scalability in enterprise healthcare settings, the serving container was deployed to Kubernetes using both native manifests (k8s/) and a packaged Helm chart (helm/heart-disease-api/).

8.1 Kubernetes Architecture & Manifest Topology

• **Deployment (k8s/deployment.yaml):** Maintains 2 replicas with a RollingUpdate strategy (maxSurge: 1, maxUnavailable: 0) ensuring zero downtime during updates. Defines strict resource governance: requests (cpu: 100m, memory: 128Mi) and limits (cpu: 500m, memory: 512Mi). Configures livenessProbe and readinessProbe pointing to /health.
• **Service (k8s/service.yaml):** Exposes the API externally via a LoadBalancer service mapping port 80 to container port 8000.
• **Ingress (k8s/ingress.yaml):** NGINX Ingress resource routing HTTP requests to the backend service.
• **Autoscaling (k8s/hpa.yaml):** Horizontal Pod Autoscaler dynamically scales pods between 2 and 5 replicas when average CPU utilization exceeds 70%.

8.2 Helm Chart Packaging & Local Deployment Verification

The Helm chart (helm/heart-disease-api/) parameterizes image repositories, replicas, resource limits, and ingress hosts in values.yaml. The system was verified on local Kubernetes (Minikube / Docker Desktop) using the following deployment sequence:

```
Kubernetes Execution Sequence:
1. Build & tag image: docker build -t heart-disease-api:latest .
2. Apply manifests: kubectl apply -f k8s/deployment.yaml -f k8s/service.yaml -f k8s/hpa.yaml
3. Verify rollout: kubectl rollout status deployment/heart-disease-api
4. Inspect pods: kubectl get pods -l app=heart-disease-api -o wide
5. Port-forward & test: kubectl port-forward svc/heart-disease-service 8000:80
6. Run verification suite: python scripts/test_serving.py http://localhost:8000
```

9. Observability, Model Governance & Conclusion

9.1 Production Telemetry with Prometheus & Grafana: Production ML systems must monitor both infrastructure health and machine learning operational metrics. Telemetry is collected via prometheus-fastapi-instrumentator and a custom monitoring stack (monitoring/docker-compose.monitoring.yml):

• **Custom ML Metrics:** heart_disease_predictions_total counter partitioned by clinical outcome (disease vs no_disease) and heart_disease_inference_latency_seconds histogram capturing latency distributions.
• **Grafana Dashboard (monitoring/dashboards/heart_disease_dashboard.json):** Pre-configured visualization panels displaying real-time Requests Per Second (RPS), 95th-percentile inference latency, positive-to-negative prediction ratios, and HTTP error rates.

9.2 Data Drift & Concept Drift Governance

In clinical cardiology deployment, patient populations experience temporal shifts in age distribution, smoking prevalence, or diagnostic criteria. The production governance strategy includes: (1) Logging feature input distributions periodically to detect covariate shift; (2) Comparing monthly prediction class ratios against training baselines (45.9% positive); (3) Setting automated alert thresholds in Grafana when latency exceeds 250ms or when positive prediction rates deviate by >15%.

9.3 Conclusion & Deliverables Compliance Summary

All nine assignment tasks spanning 50 marks have been rigorously implemented, verified, and packaged into a production-ready codebase:
1. **Data & EDA (5 marks):** Automated download, cleaning 6 missing values, and publication-quality visualization plots.
2. **Feature Engineering & Modeling (8 marks):** Scikit-Learn ColumnTransformer pipeline, 3 classifiers, 5-fold CV, 88.52% test accuracy.
3. **Experiment Tracking (5 marks):** MLflow SQLite backend tracking hyperparameters, cross-validation metrics, and plots.
4. **Model Packaging (7 marks):** Serialized .joblib pipeline bundling preprocessor and model, pinned requirements.txt.
5. **CI/CD Pipeline (8 marks):** 12 Pytest tests passing, GitHub Actions workflow with linting, testing, training, and Docker verification.
6. **Containerization (5 marks):** Production Dockerfile, non-root user, FastAPI /predict and /health endpoints.
7. **Production Deployment (7 marks):** Kubernetes Deployment, Service, Ingress, HPA manifests, and packaged Helm chart.
8. **Monitoring & Logging (3 marks):** Structured JSON request logs, Prometheus /metrics, and Grafana dashboard.
9. **Documentation & Report (2 marks):** Complete README, architecture diagrams, this 10-page formal academic report, and video script.

Academic Integrity & Student Confirmation: This implementation represents original, independent engineering effort conforming strictly to BITS Pilani academic integrity guidelines. Repository code, configurations, and documentation were created exclusively for AIMLCZG523 Assignment 01.